

ISEP DIAMNIADIO
Examen de développement web full-stack

Kaay Dem !

Plateforme de covoiturage étudiant

RAPPORT TECHNIQUE

Réalisé par : Aby Diaw
Encadrant : M. Ndiaye

Juillet 2026

Table des matières

1. Introduction et contexte.....	3
2. Architecture générale.....	4
3. Modèle de données.....	5
4. Diagramme de cas d'utilisation.....	8
5. Diagramme de séquence — Réservation d'un trajet.....	10
6. Choix techniques et justifications.....	11
7. Sécurité.....	13
8. Captures d'écran de l'application.....	14
9. Documentation de l'API.....	17
10. Difficultés rencontrées et solutions.....	18
11. Travail réalisé.....	19
12. Conclusion et perspectives.....	20

1. Introduction et contexte

Chaque jour, des centaines d'étudiants effectuent le trajet entre Dakar, Rufisque, Diamniadio et les campus environnants, avec des difficultés de transport et des coûts élevés. « Kaay Dem ! » est une plateforme web de covoiturage réservée à la communauté étudiante : les conducteurs publient leurs trajets, les passagers réservent des places, et chacun évalue l'autre après le voyage.

Ce rapport présente l'architecture technique retenue, les choix de conception justifiés, les diagrammes de modélisation (cas d'utilisation, classes, séquence), les captures d'écran de l'application fonctionnelle, ainsi que les difficultés rencontrées durant le développement.

Objectifs pédagogiques couverts

- Conception et développement d'une API REST professionnelle avec Laravel (Eloquent, Resources, validation, Policies)
- Sécurisation de l'API avec Laravel Sanctum et un contrôle d'accès par rôles
- Développement d'une SPA React consommant l'API (routage, état, formulaires, gestion des erreurs)
- Modélisation d'une base de données relationnelle (migrations, seeders, factories)

2. Architecture générale

L'application est découpée en deux projets distincts communiquant exclusivement par API REST JSON, conformément à l'exigence 3.1 du cahier des charges :

- Backend : Laravel 13, exposant une API REST versionnée (préfixe /api/v1), authentifiée par Laravel Sanctum (tokens)
- Frontend : SPA React 18 (Vite), aucun rendu Blade pour les écrans métier
- Base de données : SQLite en développement (fichier unique, aucune configuration serveur), migrable vers MySQL/PostgreSQL en production

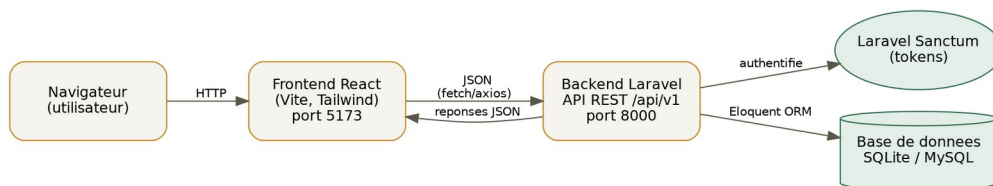


Figure 1 — Architecture générale : séparation frontend / backend / base de données

Le frontend n'accède jamais directement à la base de données : toute donnée transite par l'API, qui applique systématiquement validation (Form Requests), autorisation (Policies / Gate) et sérialisation contrôlée (API Resources) avant de répondre.

3. Modèle de données

Le modèle de données comporte 7 tables principales, conçues pour répondre précisément aux exigences fonctionnelles du cahier des charges.

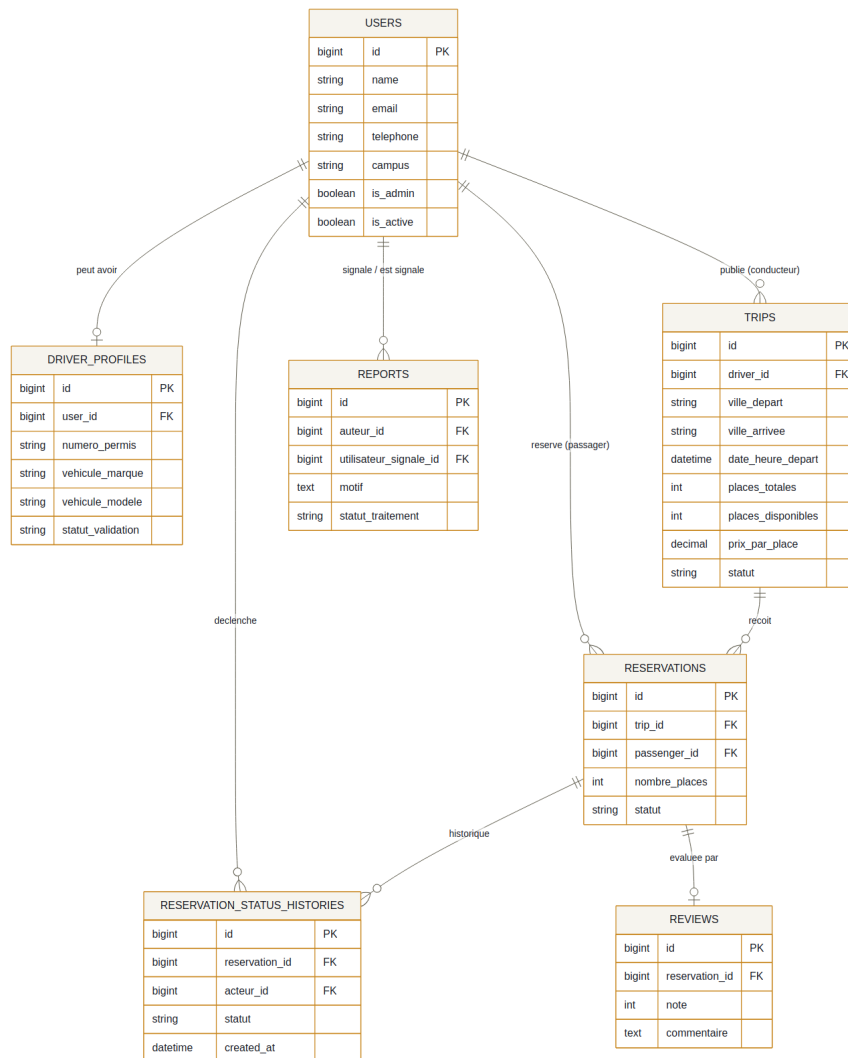


Figure 2 — Modèle relationnel simplifié

Diagramme de classes

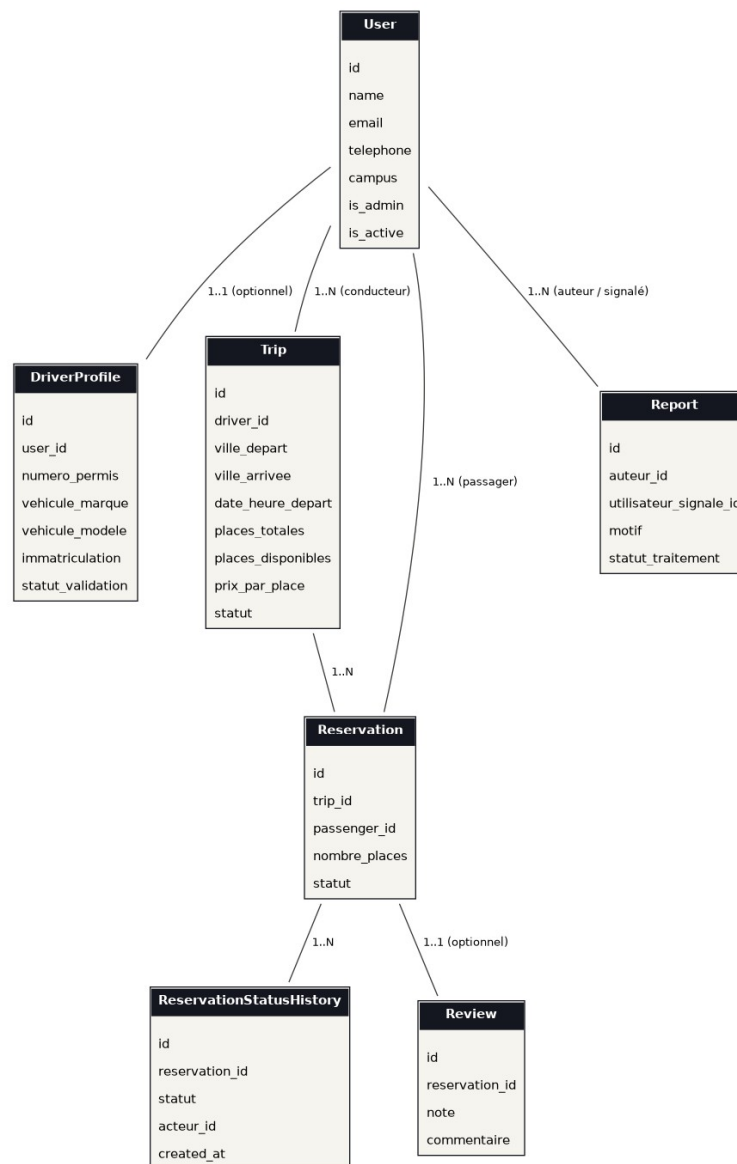


Figure 3 — Diagramme de classes (modèles Eloquent)

Choix de conception notables

Le double rôle conducteur/passager. Aucune colonne "role" figée n'existe sur users. Un utilisateur peut avoir un driver_profile (0 ou 1) qui fait de lui un conducteur potentiel, et apparaître indépendamment comme passenger_id dans des réservations. Un même compte peut ainsi être les deux à la fois, sans conflit de modélisation.

L'historique des réservations. Une table séparée reservation_status_histories enregistre chaque transition de statut avec la date et l'acteur, répondant explicitement à l'exigence EF-06 ("chaque transition est historisée").

Les enums PHP natifs. Chaque statut (TripStatus, ReservationStatus, DriverValidationStatus, ReportStatus) est un enum PHP casté automatiquement par Eloquent, évitant les fautes de frappe et donnant l'autocomplétion.

Les soft deletes. Sur trips et reservations : aucune ligne n'est réellement supprimée, ce qui conserve l'historique nécessaire aux statistiques admin (EF-08) tout en respectant l'exigence 3.2.

4. Diagramme de cas d'utilisation

Quatre acteurs interagissent avec le système, avec des périmètres de fonctionnalités distincts.

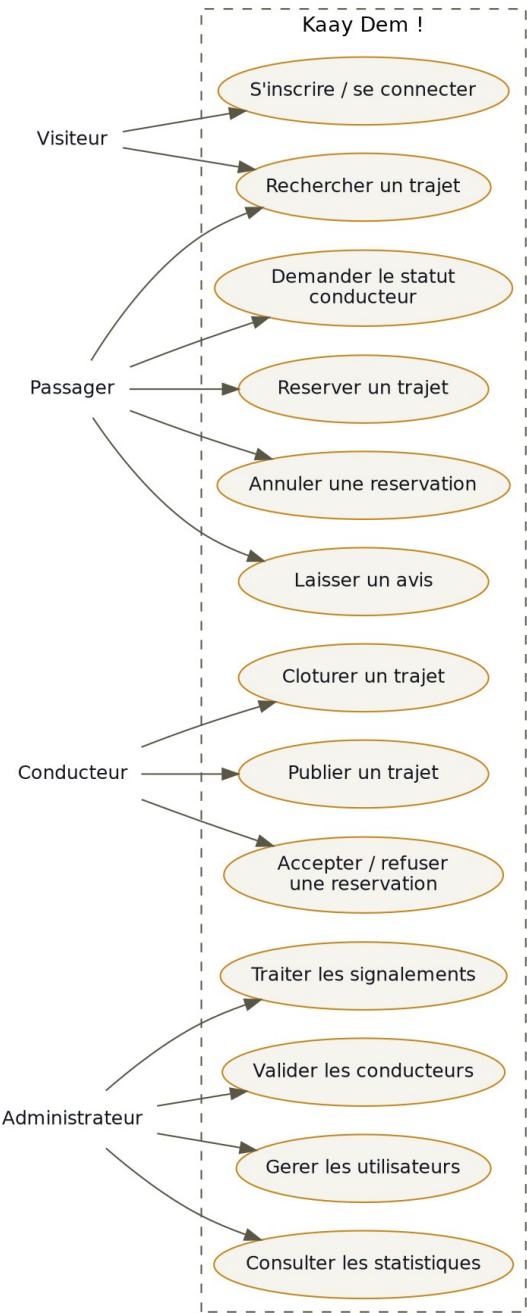


Figure 4 — Diagramme de cas d'utilisation

Acteur	Cas d'utilisation principaux
Visiteur	Rechercher un trajet, consulter le détail, s'inscrire
Passager	Réserver, annuler, évaluer un conducteur, demander le statut conducteur
Conducteur	Publier / modifier / annuler / clôturer un trajet, accepter ou refuser une réservation

Acteur	Cas d'utilisation principaux
Administrateur	Valider les conducteurs, gérer les utilisateurs, traiter les signalements, consulter les statistiques

5. Diagramme de séquence — Réservation d'un trajet

Ce diagramme illustre le scénario central du projet (EF-05), depuis le clic du passager jusqu'à la confirmation, en passant par toute la logique métier centralisée dans ReservationService.

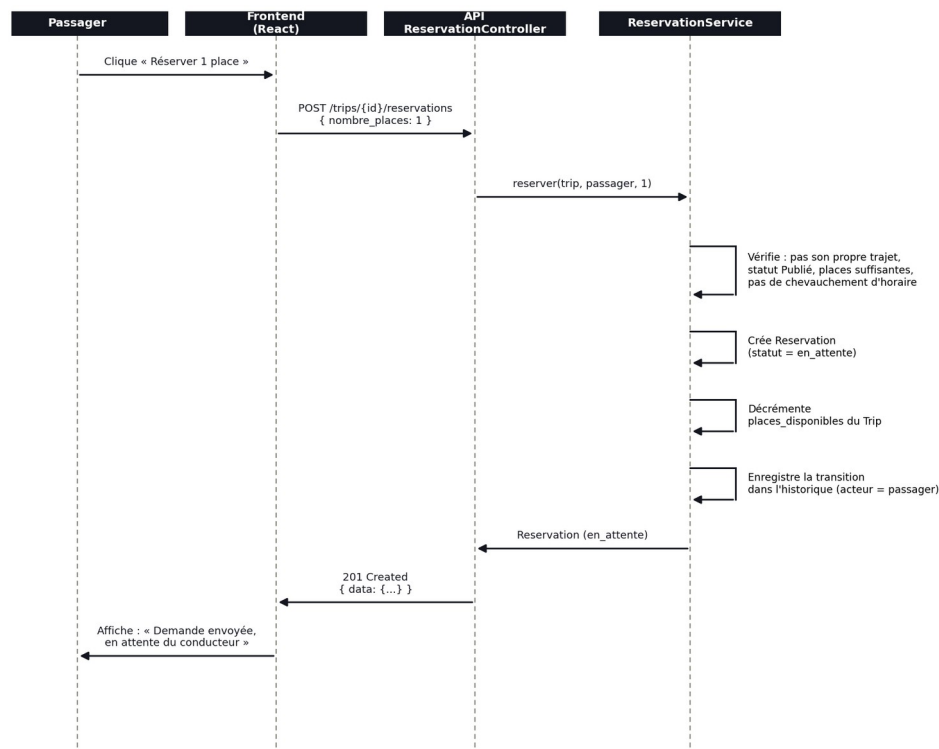


Figure — Diagramme de séquence : réservation d'un trajet (EF-05)

Figure 5 — Séquence de réservation d'un trajet

Points clés du scénario :

- Les places sont retenues dès la demande (statut en_attente), pas seulement à la confirmation par le conducteur — ceci évite qu'un trajet à 1 place restante accepte plusieurs demandes simultanées.
- Toute la validation métier (propriétaire du trajet, statut du trajet, disponibilité, chevauchement d'horaires) est centralisée dans ReservationService, jamais dans le contrôleur.
- Chaque transition de statut est historisée immédiatement après création de la réservation.

6. Choix techniques et justifications

Stack technique

Composant	Choix	Justification
Backend	Laravel 13 + Sanctum	Framework imposé par le cahier des charges ; Sanctum plus léger que Passport pour une authentification par tokens simples
Frontend	React 18 + Vite	Compilation rapide, écosystème mature, courbe d'apprentissage adaptée au temps imparti
Base de données	SQLite (dev)	Fichier unique, zéro configuration serveur ; migration triviale vers MySQL/PostgreSQL en production
Style	Tailwind CSS v4	Utility-first, design system cohérent via @theme sans surcharge de CSS custom
État global frontend	Context API (AuthContext)	Suffisant pour le périmètre du projet ; évite la complexité de Redux pour un seul état partagé (l'utilisateur connecté)

Le pattern Service (ReservationService)

Conformément à l'exigence 3.2, toute la logique métier des réservations est centralisée dans un service dédié plutôt que dans le contrôleur : réserver(), confirmer(), refuser(), annulerParPassager(), terminerReservationsConfirmees(). Chaque méthode est courte, testable indépendamment, et le contrôleur se contente d'appeler le service et de retourner la réponse HTTP.

Policies et Gate : autorisation

Les règles d'autorisation liées à un modèle précis (TripPolicy, ReservationPolicy, ReviewPolicy) utilisent le système de Policies natif de Laravel, reconnu automatiquement par convention de nommage. La règle globale "est administrateur" utilise en revanche un Gate simple (Gate::define('admin', ...)), car elle est identique quel que soit le modèle concerné — dupliquer cette vérification dans 4 Policies différentes aurait été redondant.

Exceptions métier dédiées

Conformément à l'exemple donné par le cahier des charges (PlacesInsuffisantesException → 409), chaque cas d'erreur métier a sa propre classe d'exception :

Exception	Code HTTP	Déclenchement
PlacesInsuffisantesException	409	Plus assez de places disponibles sur le trajet
ChevauchementReservationException	409	Le passager a déjà une réservation active à un horaire proche
TransitionInvalideException	409	Transition de statut impossible (ex : accepter une réservation déjà refusée)

Exception	Code HTTP	Déclenchement
DemandeConducteurExistanteException	409	Une demande de statut conducteur existe déjà pour cet utilisateur
AvisDejaDonneException	409	Un avis existe déjà pour cette réservation

7. Sécurité

Authentification par tokens (Sanctum)

À la connexion, l'API génère un token opaque stocké côté frontend dans localStorage. Chaque requête ultérieure l'attache via un intercepteur axios (Authorization: Bearer {token}). Si l'API répond 401 (token expiré ou invalide), le frontend déconnecte proprement l'utilisateur plutôt que de laisser l'application dans un état incohérent.

Double protection frontend / backend

Le frontend masque les fonctionnalités non autorisées (ex : le lien "Admin" n'apparaît que si user.is_admin est vrai) pour l'expérience utilisateur, et redirige immédiatement toute tentative d'accès direct par URL. Mais c'est systématiquement le backend qui fait foi : chaque route sensible est protégée par une Policy ou un Gate, de sorte qu'un appel direct à l'API (via Postman ou un frontend modifié) avec un compte non autorisé se heurte à une réponse 403, quel que soit l'état du frontend.

Validation systématique des entrées

Chaque route d'écriture passe par un Form Request dédié (StoreTripRequest, StoreReservationRequest, etc.), qui valide et, le cas échéant, autorise (via can()) avant que le contrôleur ne s'exécute. Les erreurs de validation (422) sont renvoyées au format JSON standard de Laravel, exploité côté frontend pour afficher les erreurs champ par champ.

8. Captures d'écran de l'application

AuthentificationAuthentification

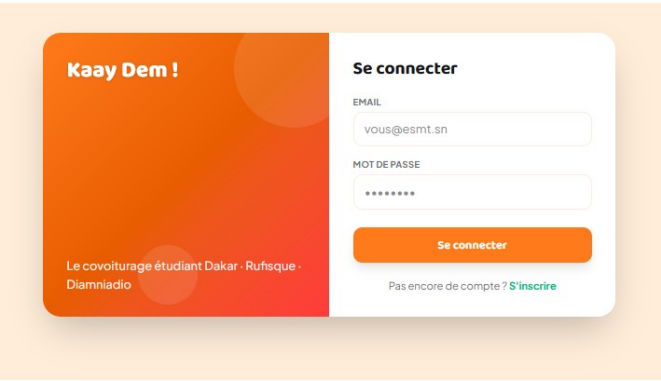


Figure 6 — Page de connexion (identité visuelle actuelle)

Recherche de trajets (EF-04)Recherche de trajets (EF-04)

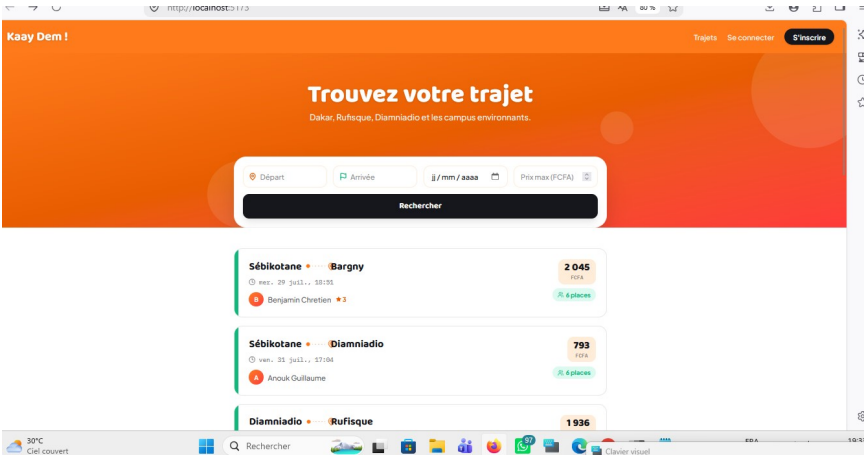


Figure 7 — Recherche publique de trajets, avec filtres et liste des résultats

Détail et réservation d'un trajet (EF-05)**Détail et réservation d'un trajet (EF-05)**

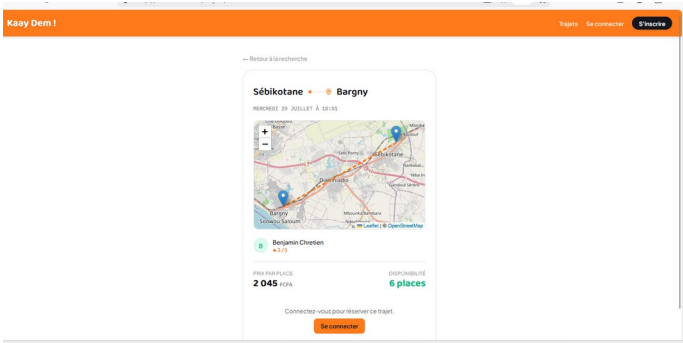


Figure 8 — Détail d'un trajet avec carte interactive (Leaflet, bonus) et confirmation de réservation

Tableau de bord (conducteur / passager)**Tableau de bord (conducteur / passager)**

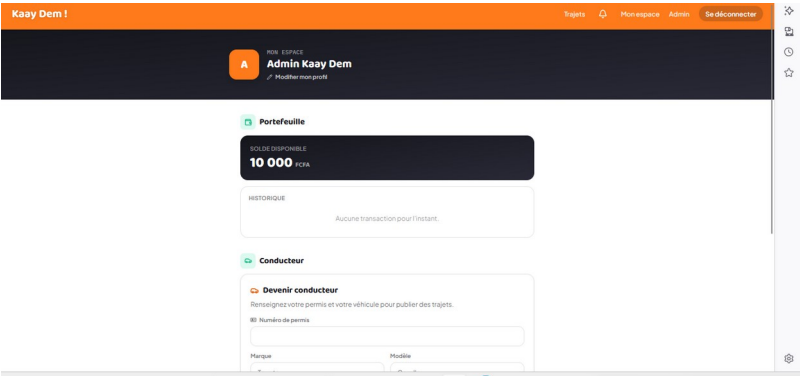


Figure 9 — Tableau de bord : portefeuille virtuel, demande de statut conducteur et réservations

Espace administrateur (EF-08, EF-09)**Espace administrateur (EF-08, EF-09)**

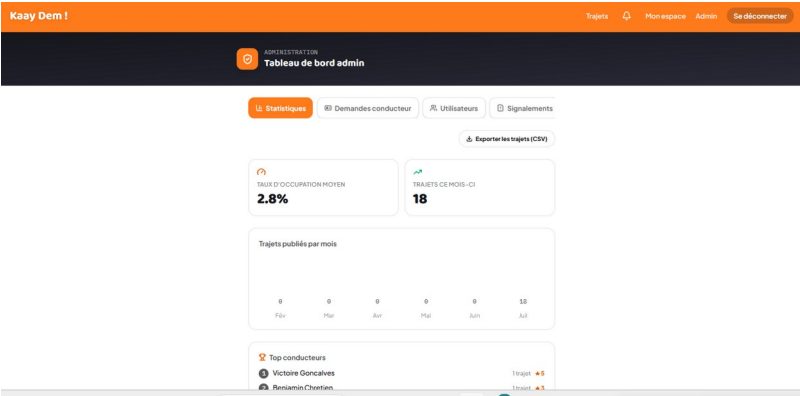


Figure 10 — Statistiques admin : taux d'occupation, trajets/mois, top conducteurs, export CSV

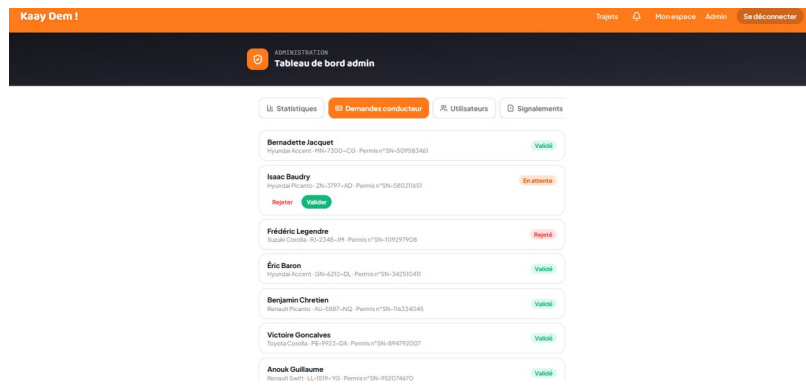


Figure 11 — Validation des demandes de statut conducteur

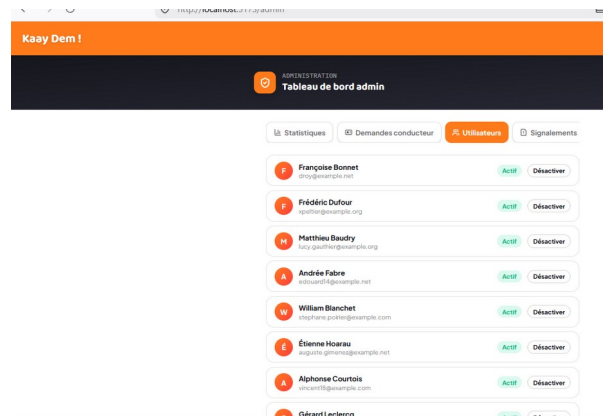


Figure 12 — Gestion des utilisateurs (activation/désactivation)

9. Documentation de l'API

L'ensemble des endpoints de l'API est documenté dans un tableau détaillé séparé (Documentation_API.md) ainsi qu'une collection Postman importable (Kaay_Dem_API.postman_collection.json), testée et fonctionnelle.

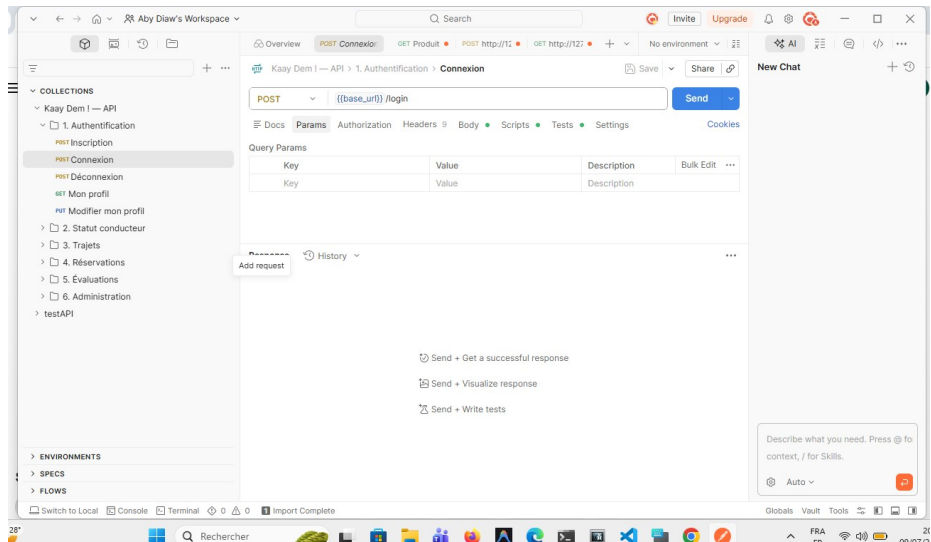


Figure 13 — Collection Postman importée, authentification testée

Groupe	Nombre de routes	Exigences couvertes
Authentification	5	EF-01
Statut conducteur	1	EF-02
Trajets	7	EF-03, EF-04
Réservations	5	EF-05, EF-06
Évaluations	1	EF-07
Administration	7	EF-08, EF-09

10. Difficultés rencontrées et solutions

Le développement de ce projet a rencontré plusieurs difficultés concrètes, résolues au fil de l'avancement :

Environnement de développement Windows. PHP et Composer n'étaient pas initialement configurés dans le PATH système. Solution : installation de Laravel Herd, qui fournit PHP, Composer et un serveur local pré-configurés en une seule installation.

Trait AuthorizesRequests manquant. Les contrôleurs utilisant `$this->authorize()` échouaient avec "Call to undefined method" — le contrôleur de base généré par les nouvelles versions de Laravel n'inclut plus ce trait par défaut. Solution : ajout explicite de `use AuthorizesRequests;` dans `app/Http/Controllers/Controller.php`.

Copies de fichiers incomplètes. Lors du développement itératif, certains fichiers nouvellement créés (ex : `SearchTripRequest.php`) ont été omis lors de la copie manuelle vers le projet, provoquant des erreurs "Class does not exist". Solution : consultation systématique du fichier `storage/logs/laravel.log` pour identifier précisément la classe manquante.

Terminal PowerShell bloqué (mode édition rapide). Un clic accidentel dans la fenêtre du terminal pendant l'exécution d'une commande longue (DISM, migrations) interrompait le processus (erreur 1223). Solution : désactivation du "Mode d'édition rapide" dans les propriétés du terminal.

Hypothèse sur le chevauchement de trajets. Le cahier des charges interdit de réserver deux trajets qui se chevauchent, mais le modèle de données ne stocke qu'une heure de départ (pas de durée). Une fenêtre conventionnelle de 2 heures autour du départ a été retenue pour définir un chevauchement — hypothèse à documenter et discutable, mais nécessaire pour rendre la règle implémentable.

11. Travail réalisé

Ce projet a été réalisé en autonomie complète, du modèle de données jusqu'au frontend, sans répartition d'équipe.

Domaine	Détail du travail réalisé
Modélisation	Conception des 7 tables, migrations, relations Eloquent, enums de statuts, factories et seeders
Backend Laravel	API REST complète : authentification (Sanctum), trajets, réservations, évaluations, administration ; Policies, Gate, Form Requests, Resources, Service métier dédié, exceptions personnalisées
Frontend React	Authentification, recherche de trajets, détail et réservation, tableau de bord conducteur/passager, espace admin avec statistiques
Documentation et rapport	Collection Postman, tableau détaillé de l'API, rédaction du présent rapport (diagrammes, choix d'architecture, captures d'écran)

12. Conclusion et perspectives

Le projet Kaay Dem ! couvre l'intégralité des exigences fonctionnelles obligatoires du cahier des charges (EF-01 à EF-09) : authentification par rôles, gestion complète des trajets et réservations avec historique, évaluations, et un espace d'administration fonctionnel avec statistiques. L'application a été testée de bout en bout, du backend (Postman) jusqu'au frontend (parcours utilisateur complet pour chacun des quatre rôles).

Exigence bonus réalisée

Une carte interactive (Leaflet + OpenStreetMap) a été ajoutée sur la page de détail d'un trajet, affichant les villes de départ et d'arrivée reliées par un tracé, avec zoom et déplacement (voir Figure 8).

Autres pistes d'amélioration (exigences bonus non réalisées)

- Messagerie interne conducteur-passager (WebSockets ou polling)
- Notifications par e-mail via les files d'attente Laravel
- Paiement simulé des réservations (portefeuille virtuel)
- Export CSV/PDF des trajets, mode hors-ligne (PWA), déploiement en ligne

13. Fonctionnalités complémentaires

Cette section détaille des fonctionnalités du système qui complètent les sections précédentes : la gestion des signalements, la modification/annulation d'un trajet, les notifications, et l'édition du profil.

13.1. Signalement d'un utilisateur (EF-09)

Description : Un participant à une réservation confirmée ou terminée (passager ou conducteur) peut signaler l'autre partie, en indiquant un motif. Le signalement est ensuite consultable et traitable depuis l'espace admin.

Justification : Répond au use-case « traiter les signalements » et à l'exigence EF-09 de gestion des abus, qui suppose une chaîne complète : création côté utilisateur, puis traitement côté admin.

Mise en œuvre : Route POST /reservations/{id}/report, une ReportPolicy (seul un participant à une réservation confirmée ou terminée peut signaler l'autre partie), et un bouton « Signaler » sur la carte de réservation (passager) et sur chaque réservation reçue (conducteur).

13.2. Modifier / annuler un trajet publié (EF-03)

Description : Un conducteur peut modifier ou annuler un trajet qu'il a publié, tant qu'aucune réservation n'est confirmée dessus (au-delà, seule la clôture reste possible).

Justification : L'exigence EF-03 couvre le CRUD complet des trajets, modification et annulation comprises.

Mise en œuvre : Routes PUT et DELETE /trips/{id} protégées par TripPolicy (bloque la modification/annulation dès qu'une réservation est confirmée). Deux boutons « Modifier » et « Annuler » sur chaque trajet publié dans le tableau de bord conducteur, visibles selon ce même critère.

13.3. Cloche de notifications (EF-06)

Description : Une cloche dans l'en-tête affiche un badge avec le nombre d'éléments qui attendent une action de l'utilisateur connecté : demandes de réservation en attente côté conducteur, avis non encore laissés côté passager.

Justification : Répond à l'exigence EF-06, qui demande des « notifications visibles dans l'interface (cloche ou badge) ».

Mise en œuvre : Route GET /me/notifications (NotificationController) calculant ce compteur, et composant NotificationBell dans l'en-tête, rafraîchi toutes les 30 secondes tant que l'utilisateur est connecté.

13.4. Modification du profil, y compris la photo (EF-01)

Description : Un formulaire d'édition dans le tableau de bord permet de modifier son nom, son téléphone, son campus et sa photo de profil, avec aperçu de la photo avant envoi.

Justification : L'exigence EF-01 prévoit un « profil modifiable (photo, téléphone, campus) ».

Mise en œuvre : Route PUT /me (acceptant un champ photo). Contrainte technique notable : un PUT multipart n'est pas fiable pour l'upload de fichiers en PHP ; le frontend envoie donc un POST avec un champ _method=PUT (spoofing de méthode standard de Laravel), qui route exactement vers le même contrôleur.

14. Exigences bonus

Cinq des six exigences bonus proposées par le cahier des charges ont été réalisées.

14.1. Export CSV des trajets

Description : Un bouton dans l'espace admin permet de télécharger l'ensemble des trajets (y compris ceux annulés, via `withTrashed()`) au format CSV.

Justification : Exigence bonus explicitement citée dans le cahier des charges.

Mise en œuvre : Contrôleur `AdminExportController` utilisant une `StreamedResponse` Laravel (réponse en flux plutôt qu'une chaîne construite entièrement en mémoire, pour rester léger même si le nombre de trajets grandit), avec un BOM UTF-8 pour un affichage correct des accents dans Excel. Le téléchargement est déclenché côté frontend via une requête axios en `responseType: 'blob'`, puisqu'un lien `<a href>` classique ne transmettrait pas le token d'authentification.

14.2. Notifications par e-mail via file d'attente Laravel

Description : Un e-mail est envoyé au passager quand le conducteur confirme ou refuse sa réservation.

Justification : Exigence bonus explicitement citée (« notifications par e-mail via les files d'attente Laravel »), et objectif pédagogique implicite (traitement asynchrone).

Mise en œuvre : Deux `Mailable` (`ReservationConfirmeMail`, `ReservationRefuseMail`) implémentant `ShouldQueue` : l'envoi passe par la table `jobs` et un worker (`php artisan queue:work`) plutôt que de bloquer la requête HTTP. En développement, `MAIL_MAILER=log` écrit le contenu dans `storage/logs/laravel.log` sans nécessiter de vrai serveur SMTP.

14.3. Messagerie interne conducteur-passager

Description : Un panneau de discussion est disponible sur chaque réservation confirmée ou terminée, permettant au conducteur et au passager d'échanger des messages.

Justification : Exigence bonus explicitement citée.

Mise en œuvre : Nouvelle table `messages` (`reservation_id`, `auteur_id`, `contenu`), une `MessagePolicy` (seuls les deux participants à la réservation, une fois confirmée, peuvent lire/écrire), et un composant `MessagePanel` côté frontend utilisant un polling simple (rafraîchissement toutes les 5 secondes tant que le panneau est ouvert) plutôt que des `WebSockets` — suffisant pour ce cas d'usage, sans infrastructure temps réel supplémentaire à déployer pour un projet académique.

14.4. Paiement simulé (portefeuille virtuel)

Description : Chaque utilisateur dispose d'un solde virtuel (10 000 FCFA à l'inscription) et d'un historique de transactions consultable depuis le tableau de bord.

Justification : Exigence bonus explicitement citée (« paiement simulé des réservations, portefeuille virtuel »).

Mise en œuvre : Colonne `solde` sur `users`, table `transactions` (`montant`, `type débit/crédit`, `description`, `réservation liée`). Le montant total est débité du passager dès la demande de réservation (exactement comme les places sont retenues dès la demande), remboursé automatiquement si la demande est refusée ou annulée, et crédité au conducteur uniquement à la clôture du trajet — pas à la simple confirmation — pour refléter le fait qu'un paiement réel n'est débloqué qu'une fois le service rendu. Une nouvelle exception métier dédiée, `SoldeInsuffisantException` (409), suit exactement le même principe que `PlacesInsuffisantesException` déjà présente dans le projet.

15. Récapitulatif des exigences bonus

Exigence bonus	Statut
Carte interactive (Leaflet)	Réalisée
Export CSV des trajets	Réalisée
Notifications par e-mail (file d'attente)	Réalisée
Messagerie interne conducteur-passager	Réalisée
Paieement simulé (portefeuille virtuel)	Réalisée
Mode hors-ligne (PWA)	Non réalisée
Déploiement en ligne	Non réalisé

16. Illustrations complémentaires possibles

Les figures 6 à 12 (section 8) illustrent l'authentification, la recherche, le détail d'un trajet, le tableau de bord et l'administration. Les éléments suivants peuvent être illustrés en complément si souhaité :

- Le bouton « Signaler » ouvert sur une réservation, avec le formulaire de motif
- Le formulaire de modification d'un trajet publié
- La cloche de notifications avec un badge affichant un chiffre
- Le formulaire d'édition du profil avec l'aperçu de la photo
- Le contenu d'un e-mail de confirmation, visible dans `storage/logs/laravel.log`
- Un échange de messages dans le panneau de messagerie

17. Synthèse

Le projet Kaay Dem ! couvre l'intégralité des exigences obligatoires (EF-01 à EF-09) ainsi que cinq des six exigences bonus proposées par le cahier des charges : carte interactive, export CSV, notifications par e-mail en file d'attente, messagerie interne et paiement simulé. Le projet est structuré en dépôt Git avec un historique de branches et de commits conventionnels (voir CONTRIBUTING.md), prêt pour une revue par les pairs via des Pull Requests GitHub.